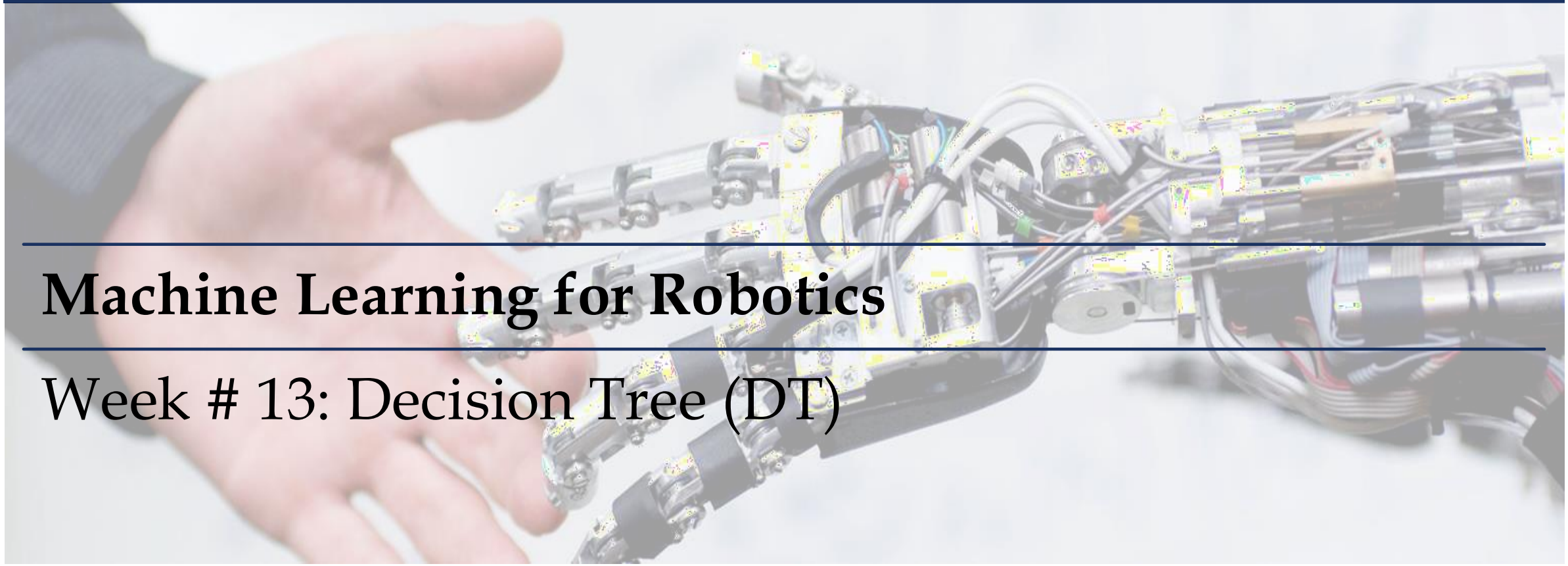


Machine Learning for Robotics

Week # 13: Decision Tree (DT)

Dr. Maryam Iqbal

ABOUT INSTRUCTOR

- PhD EE (Controls, Automation & Rehabilitation Robotics)
- Research areas: Robotics & Automation, Artificial Intelligence, Machine Learning, Biomechanics, AI in Healthcare
- In charge : BSRIS Program at Bahria University
- Research Publications, Funded Projects and Patents
- Member of AI Task Force (Ministry of Planning, Development & Special Initiatives of Pakistan)
- Senior Member IEEE (Region 10) | General Secretary IEEE RAS (Region 3) | Member IEEE CSS
- Email: maryamiqbal.buic@bahria.edu.pk
- LinkedIn: linkedin.com/in/maryam-iqbal-3b942a66
- Google Scholar: https://scholar.google.com/citations?user=-4togNoAAAAJ&hl=en

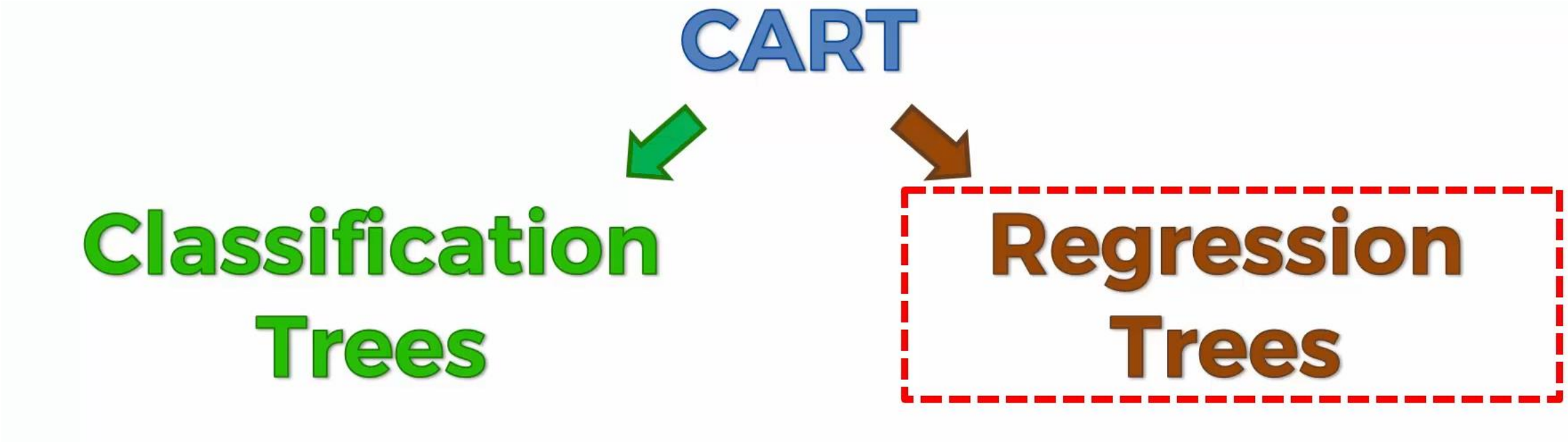
CONTENTS


DECISION TREE REGRESSION

INTRODUCTION TO DT

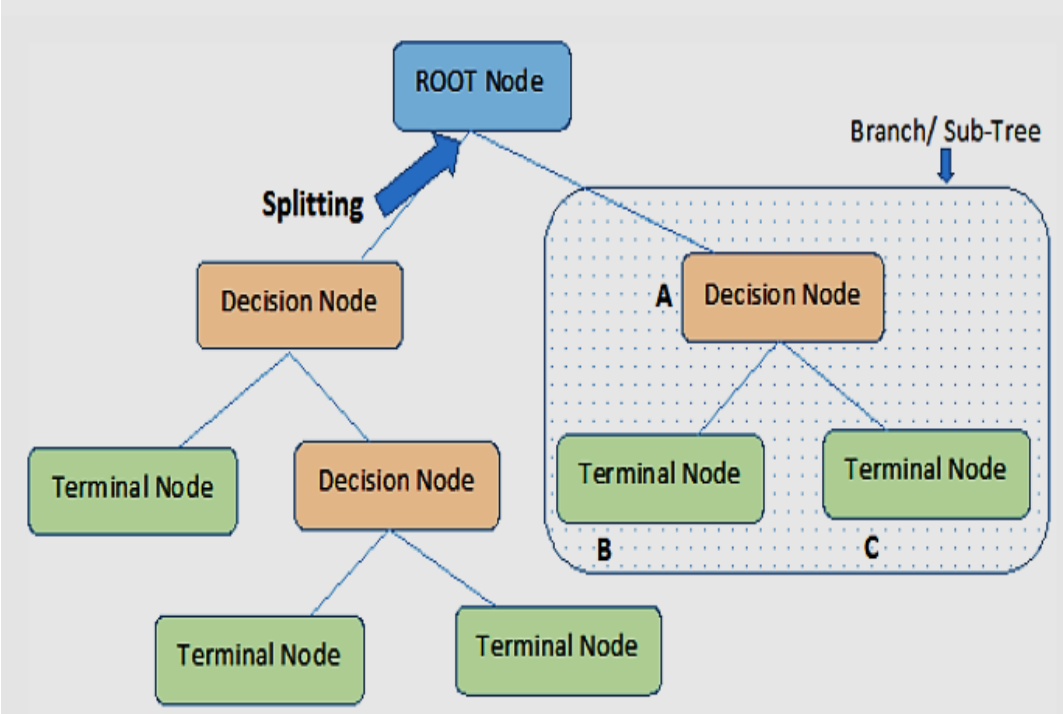
- DT is a type of supervised learning algorithm that is commonly used in machine learning to design a model and predict outcomes.
- It is a tree-like structure where each internal node tests on attribute/feature.
- Each branch corresponds to attribute value and each leaf node represents the final decision or prediction.
- Used to solve both **regression** and **classification problems**.
- The goal of using a Decision Tree is to create a training model that can use to predict the class of the target sample by **learning simple decision rules**.

CLASSIFICATION AND REGRESSION TREES



BASIC TERMS IN DT

1. **Root Node:** The starting/first node of the tree, representing the initial decision.
2. **Splitting:** It is a process of dividing a node into two or more sub-nodes.
3. **Decision Node:** When a sub-node splits into further sub-nodes, then it is called the decision node.
4. **Leaf / Terminal Node:** Nodes do not split is called Leaf or Terminal node.
5. **Pruning:** Removing of sub-nodes of a decision node is called pruning. You can say the opposite process of splitting.
6. **Branch / Sub-Tree:** A subsection of the entire tree is called branch or sub-tree.

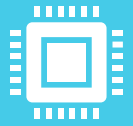


HOW DT WORKS

- The whole training set is considered as the **root**
- Feature values are preferred to be categorical like Colors (red, blue, green), sizes (small, medium, large)
- If the values are continuous then they are discretized prior to building the model
  - **Discretization** is the process of converting continuous data into discrete categories.
  - Example: A continuous variable "Age" ranging from 1 to 100 can be classified as
  - **Young:** 1-29
  - **Middle-Aged:** 30-59
  - **Old:** 60-100
- Records are **distributed recursively** on the basis of attribute values

DECISION TREE INTUITION

Tree-based methods for regression and classification involve stratifying or segmenting the predictor space into a number of simple regions.

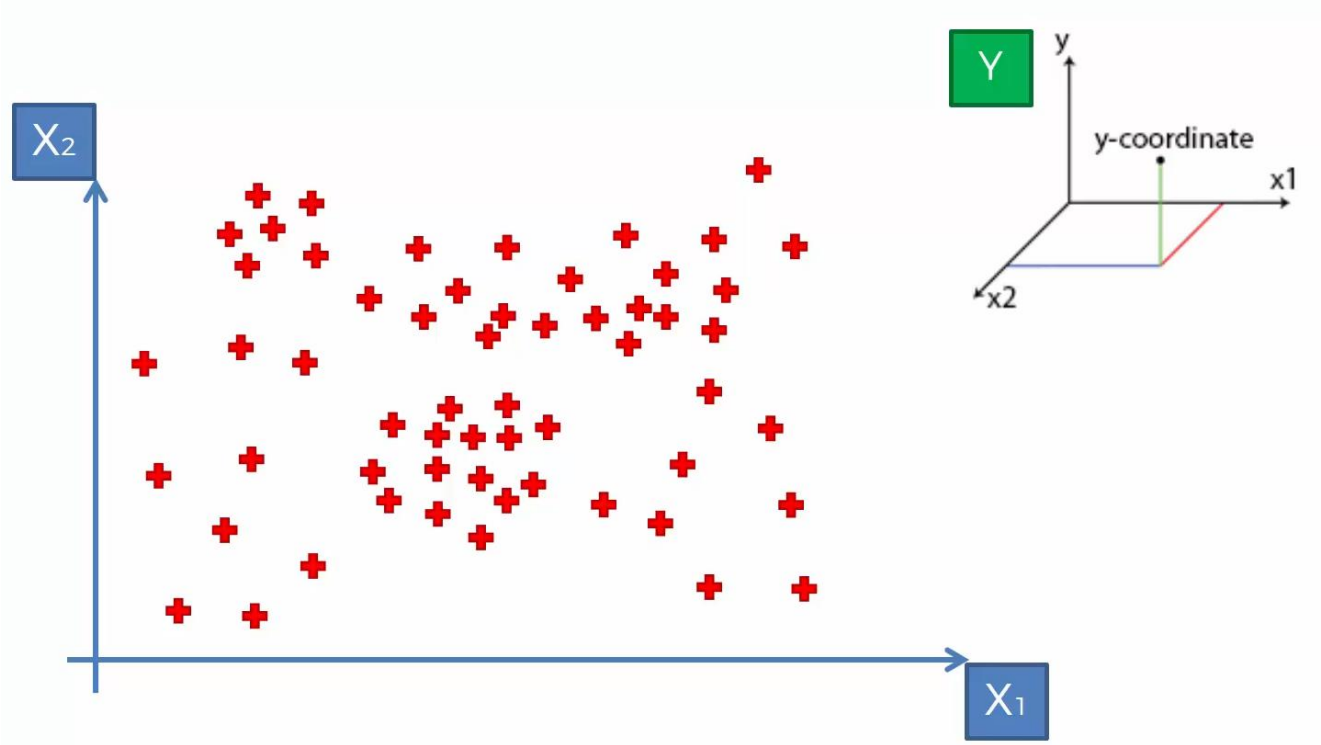


Since the set of splitting rules used to segment the predictor space can be summarized in a tree, these types of machine learning approaches are known as *decision tree* methods.

The basic idea of these methods is to partition the space and identify some representative centroids.

Decision Tree Intuition

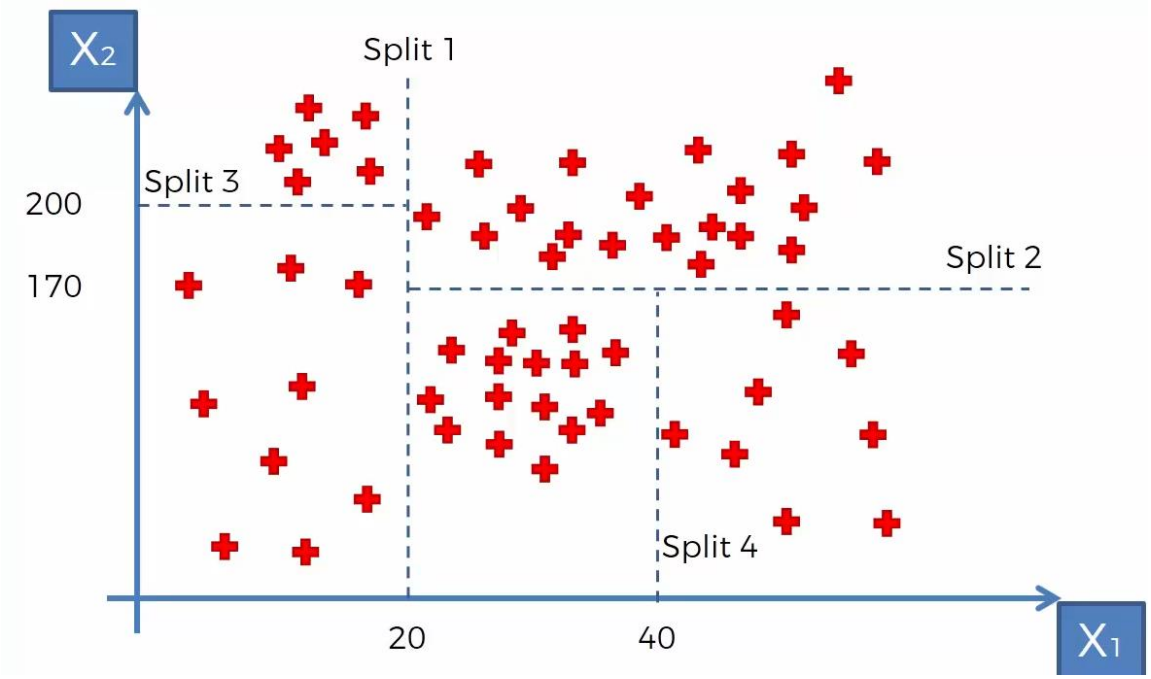
- We are given some data, which consists of two independent variable x_1 and x_2
- The plot represents a scatter plot of the data
- We want to predict a dependent variable y, which we cannot see on this scatter plot
- We will work with the data points to build a regression tree and then consider y



One way to make predictions in a regression problem is to divide the predictor space (i.e. all the possible values for X_1, X_2,…, X_p) into distinct regions, say R_1, R_2,…,R_k (*terminal nodes* or *leaves*).

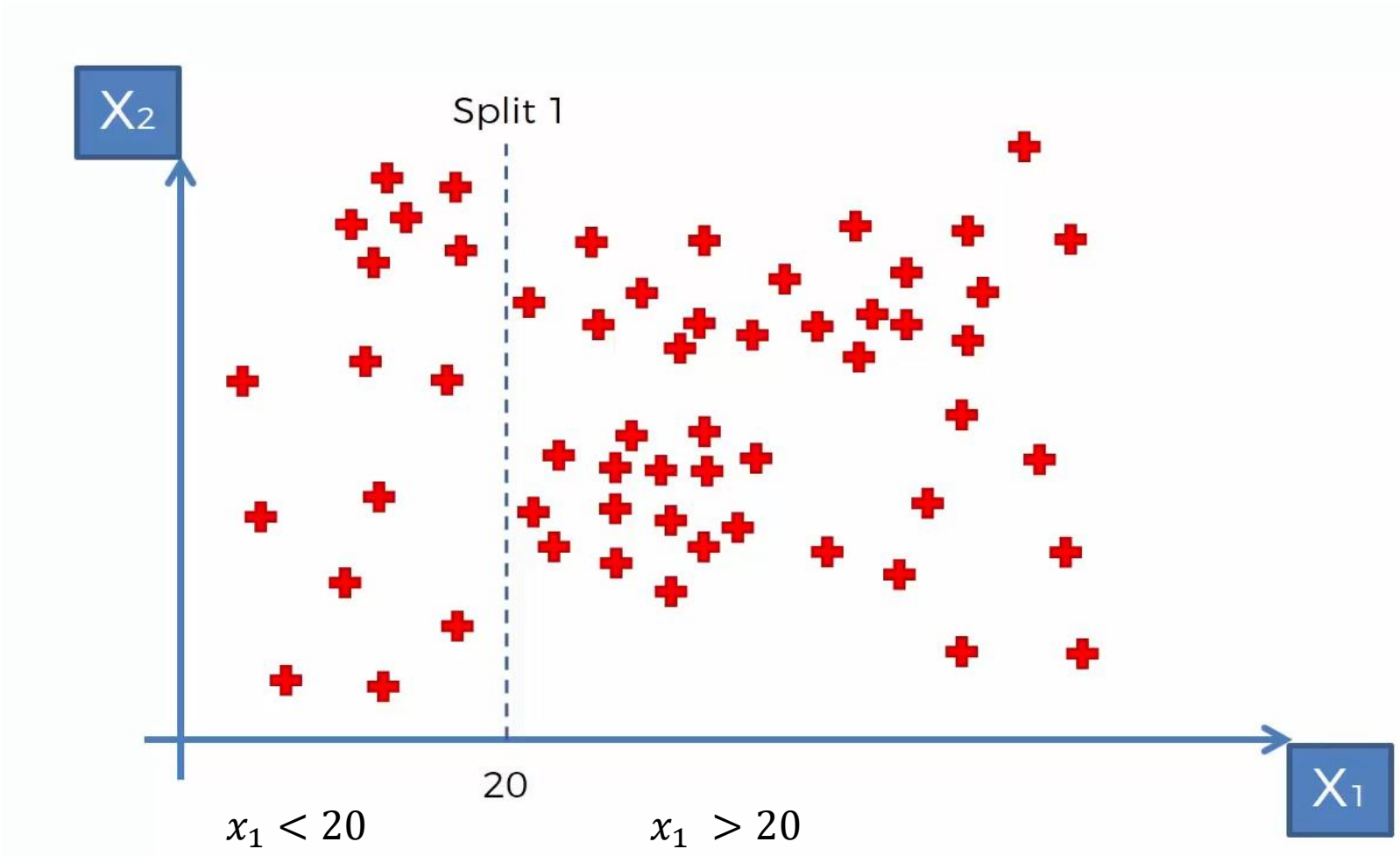
Then, for every X that falls into a particular region (say R_j) we make the same prediction.

Decision trees are typically drawn *upside down*, in the sense that the leaves are at the bottom of the tree.



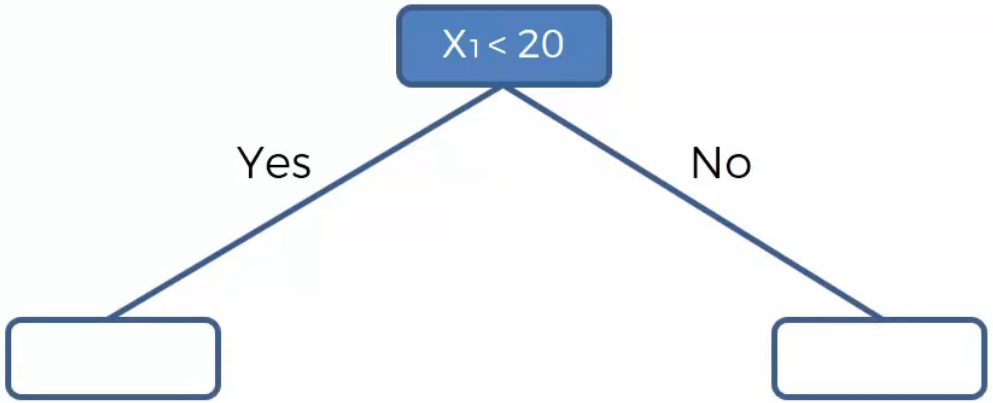
- How the data may be split?
- There needs to be a criteria over which the split takes place or not (Entropy)
- If a split adds more information to the data, then the split happens. Otherwise, the split does not happen
- The points along the tree where the predictor space is split are referred to as internal nodes.
- The terminology used for these regions created by the splits is leaf
- When a split cannot take place in a region it is called a terminal leaf

DECISION TREE ALGORITHM

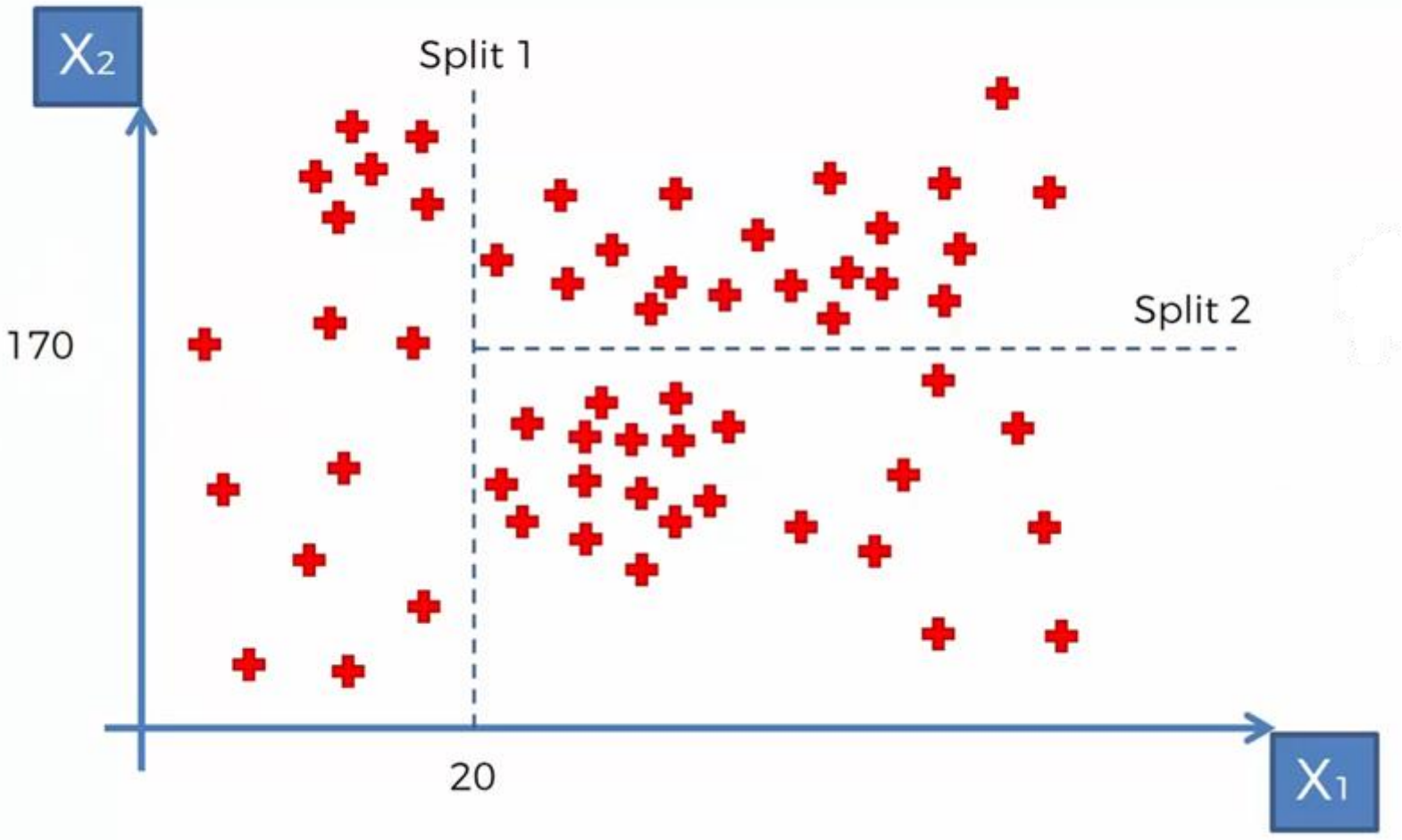


- Split 1:
  - $x_1 > 20$ *and* $x_1 < 20$

DECISION TREE ALGORITHM



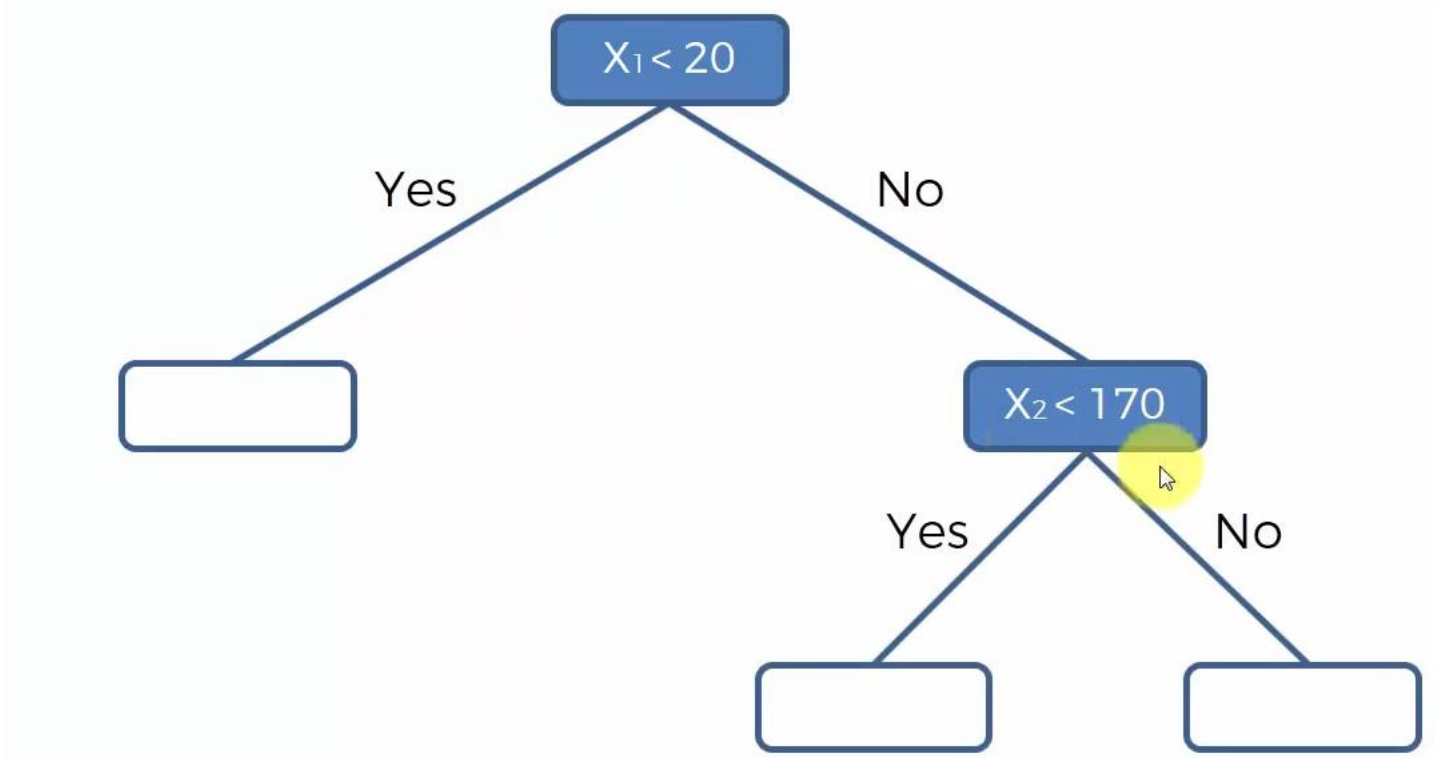
DECISION TREE ALGORITHM



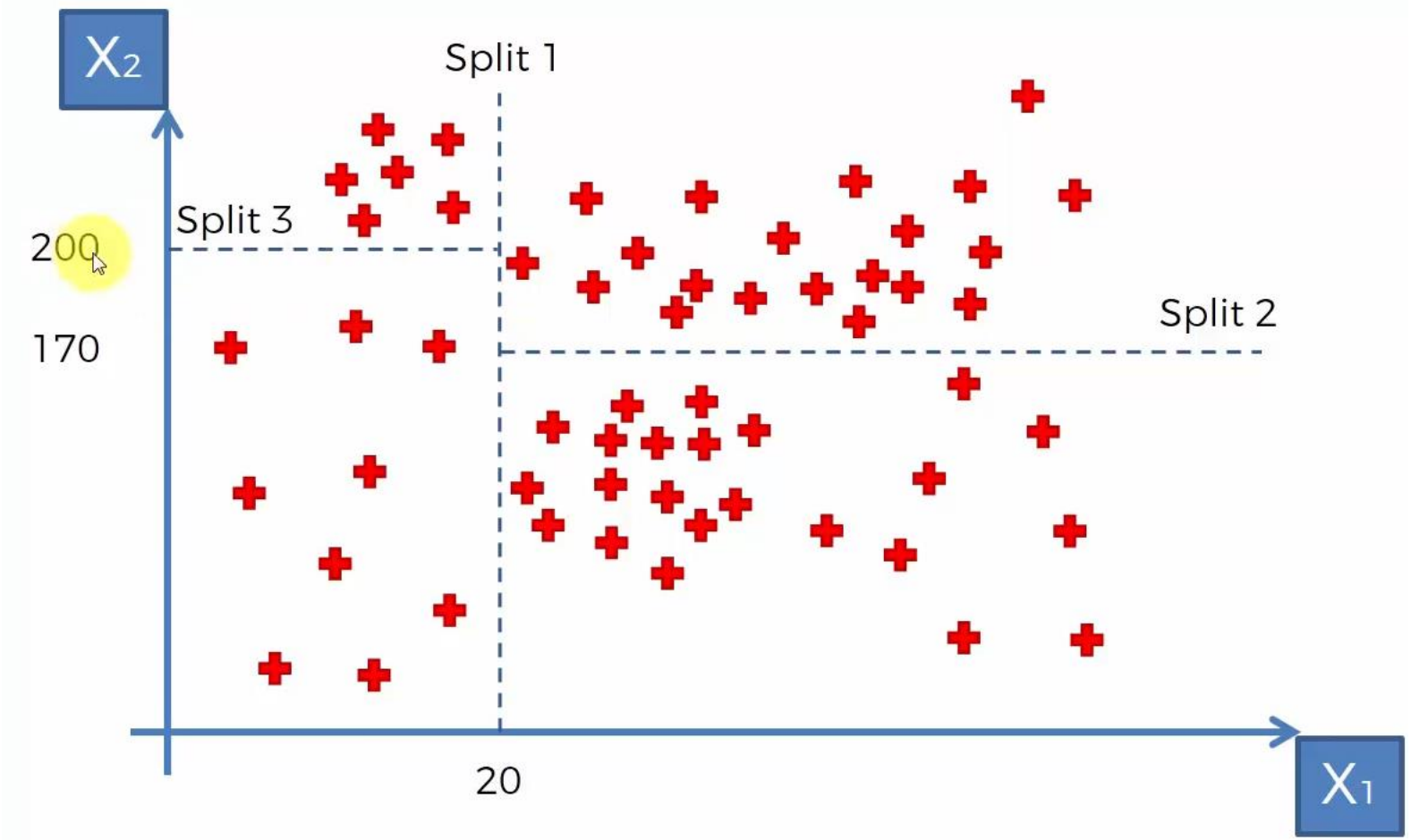
- Split 2:

$x_1 > 20$

DECISION TREE ALGORITHM

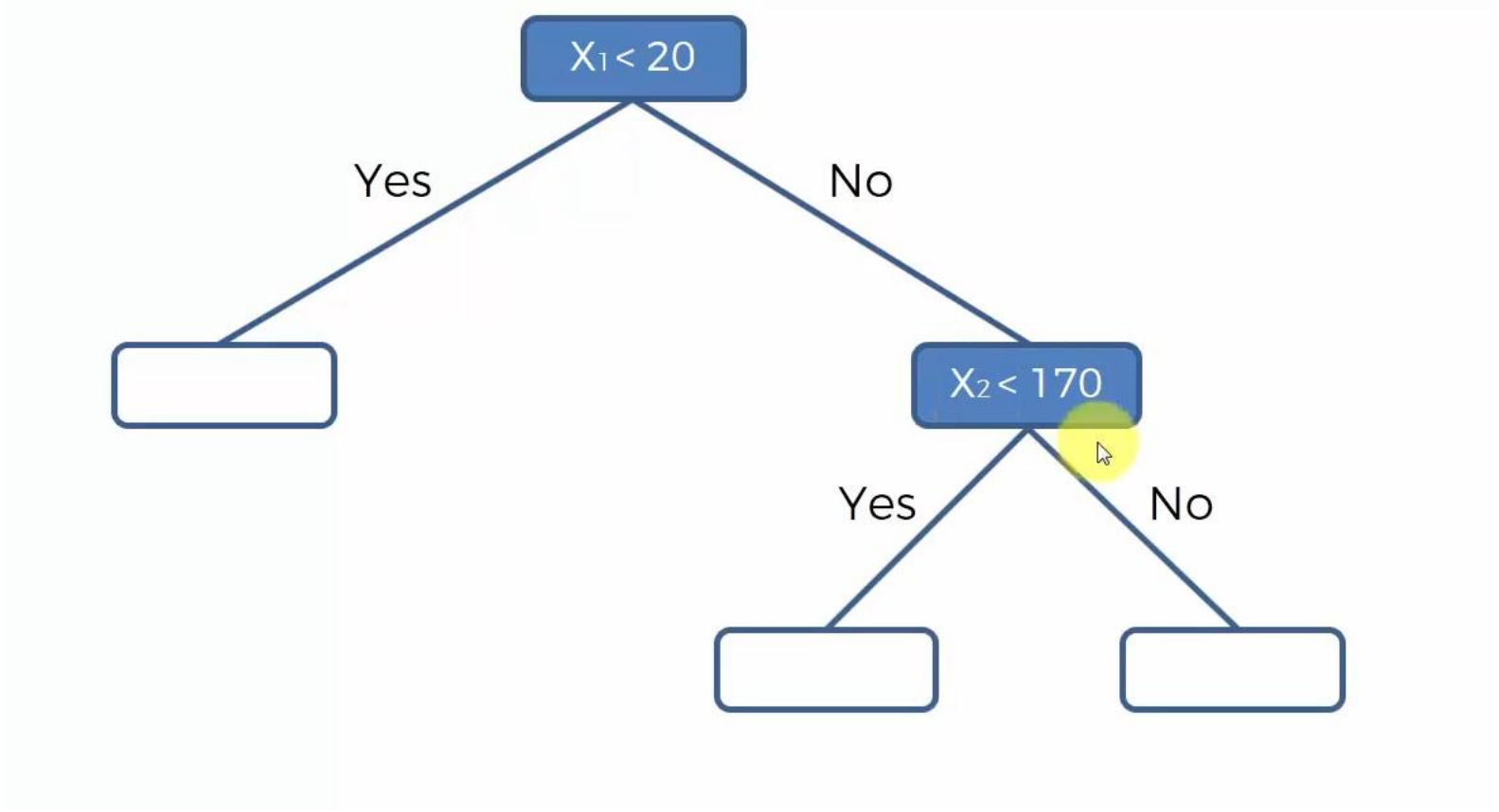


DECISION TREE ALGORITHM

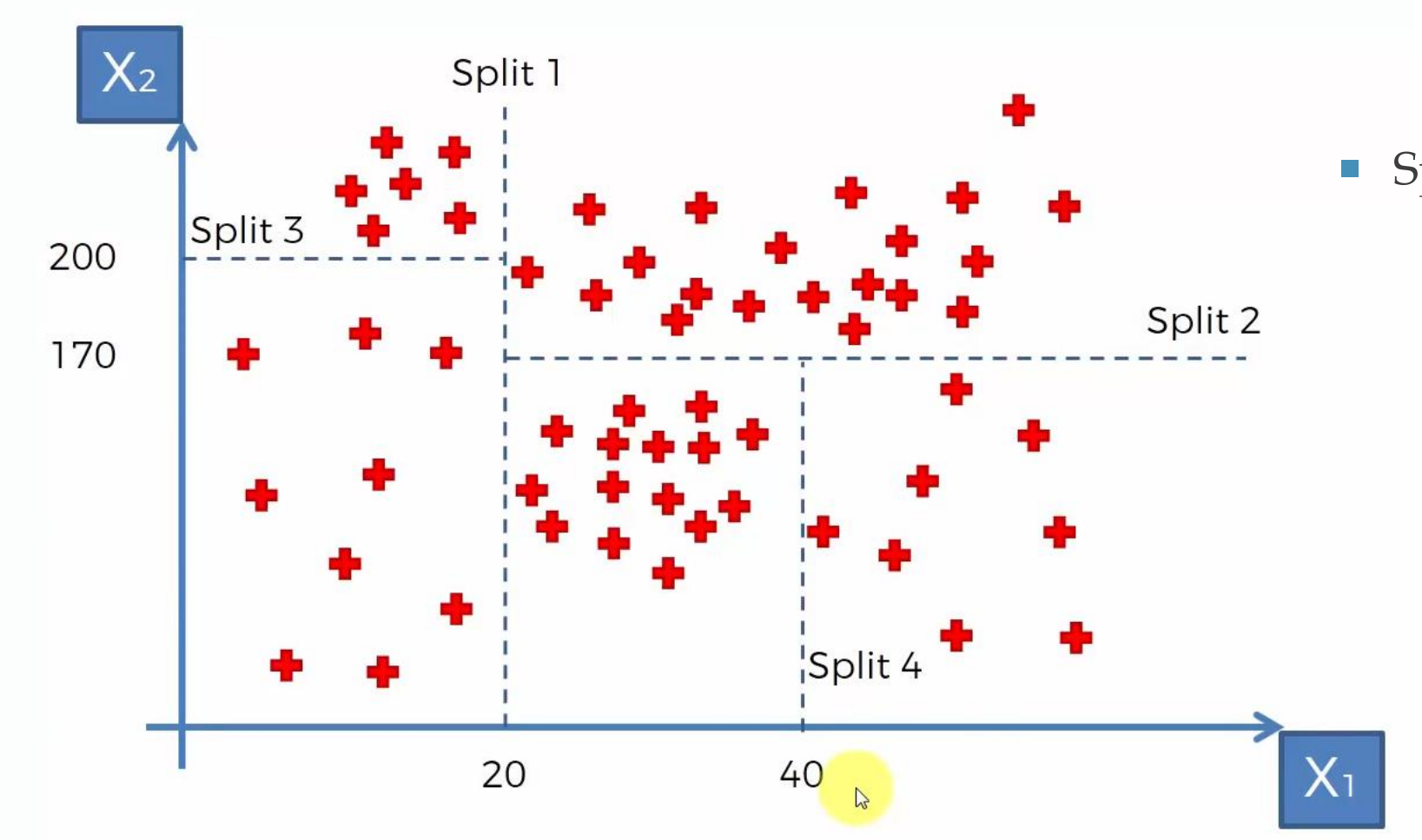


- Split 3:
  - $x_2 > 200 \; and \; x_2 < 200$
- For only those points where $x_1 < 20$

DECISION TREE ALGORITHM



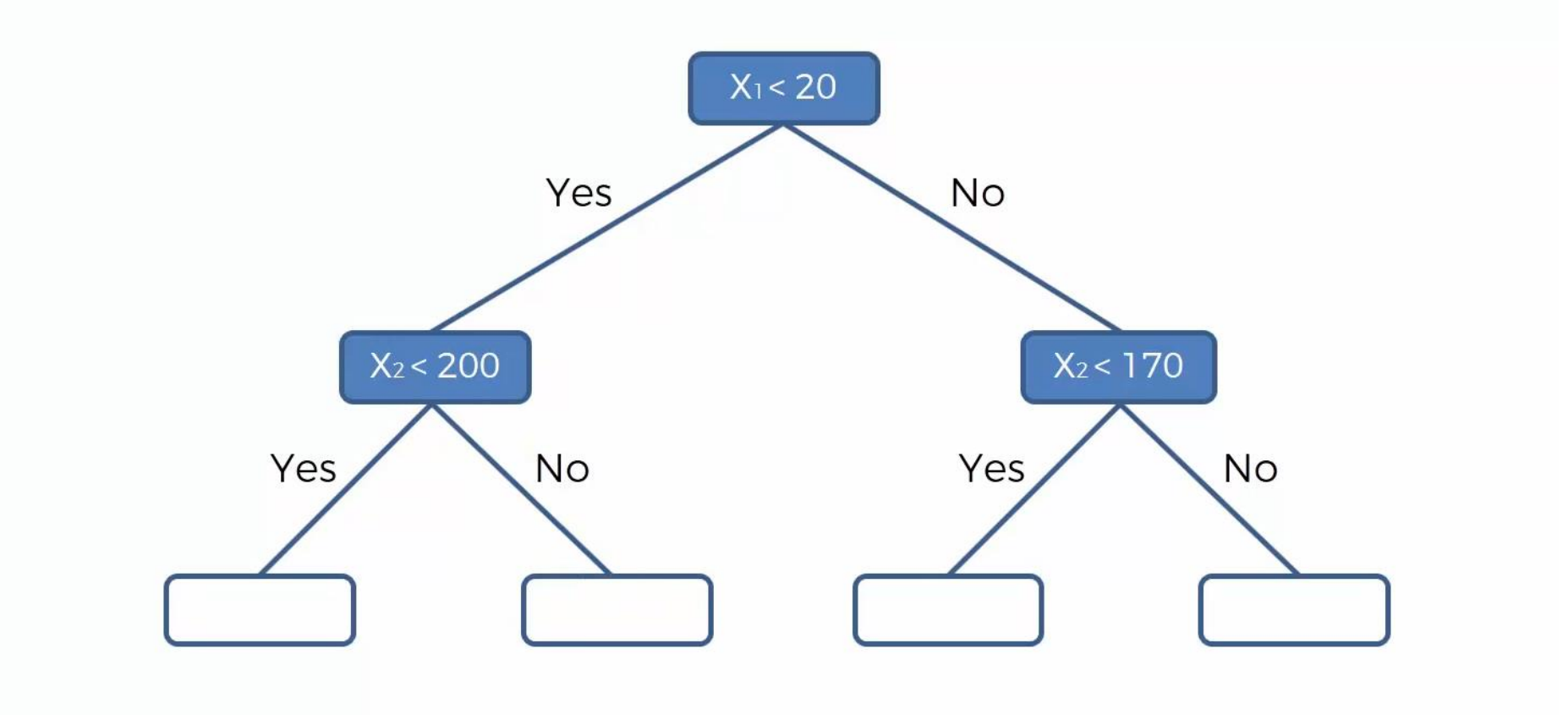
DECISION TREE ALGORITHM

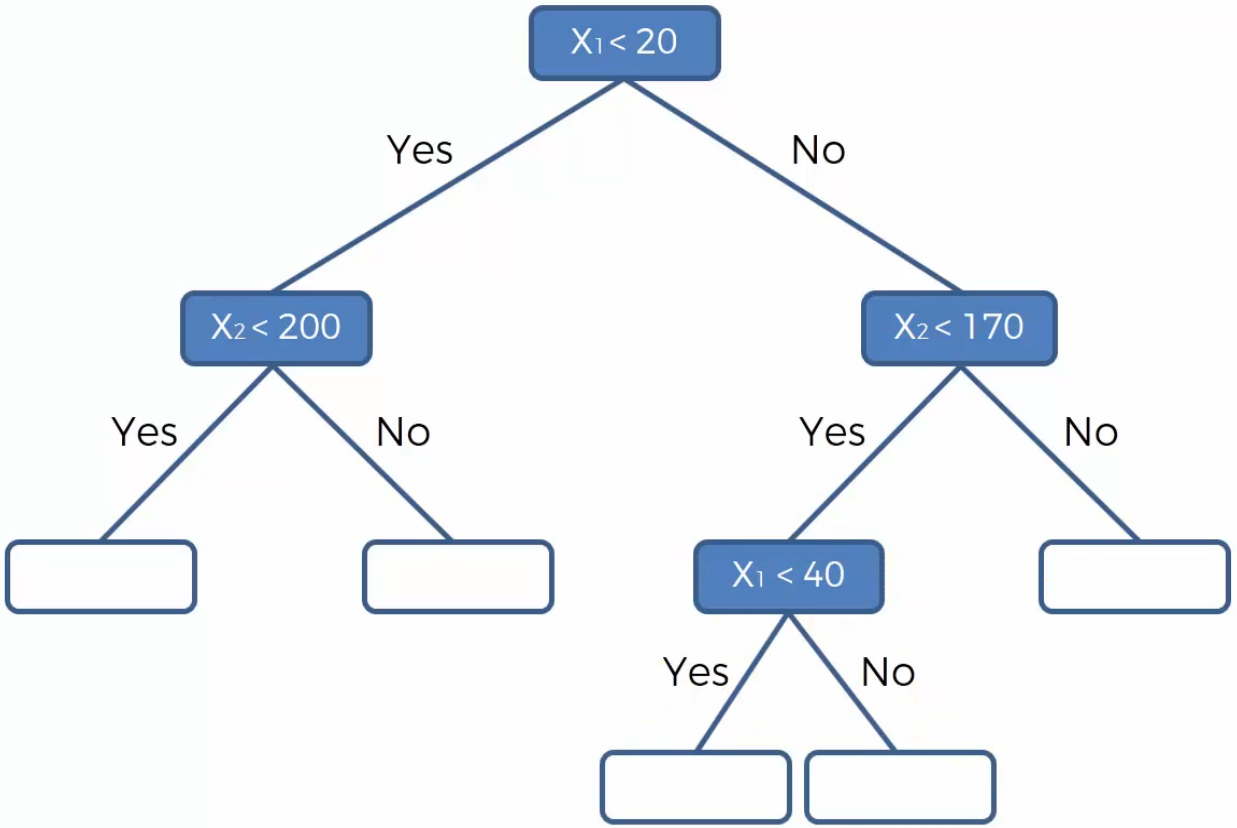


- Split 4:

$x_2 < 170$

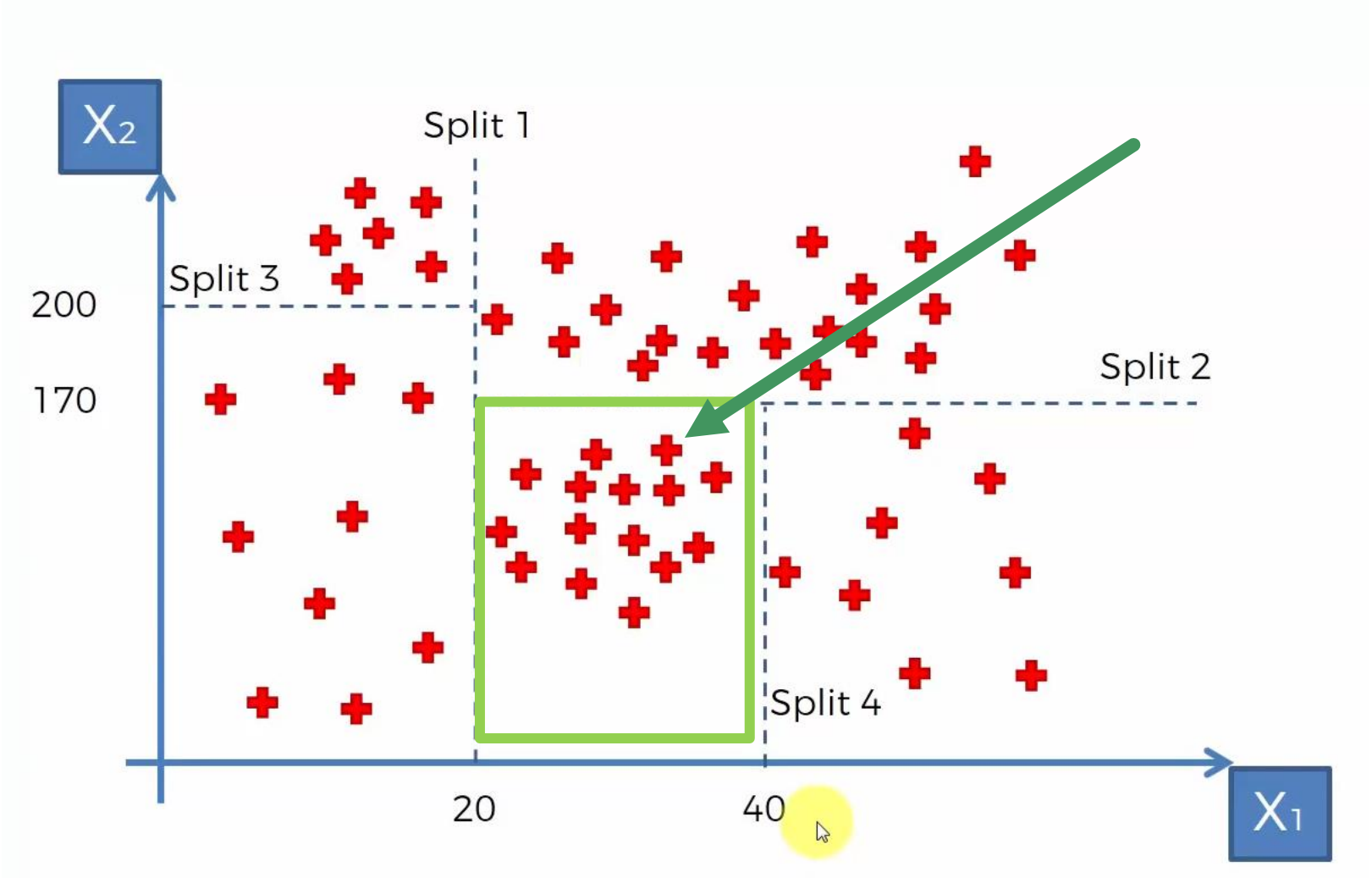
DECISION TREE ALGORITHM



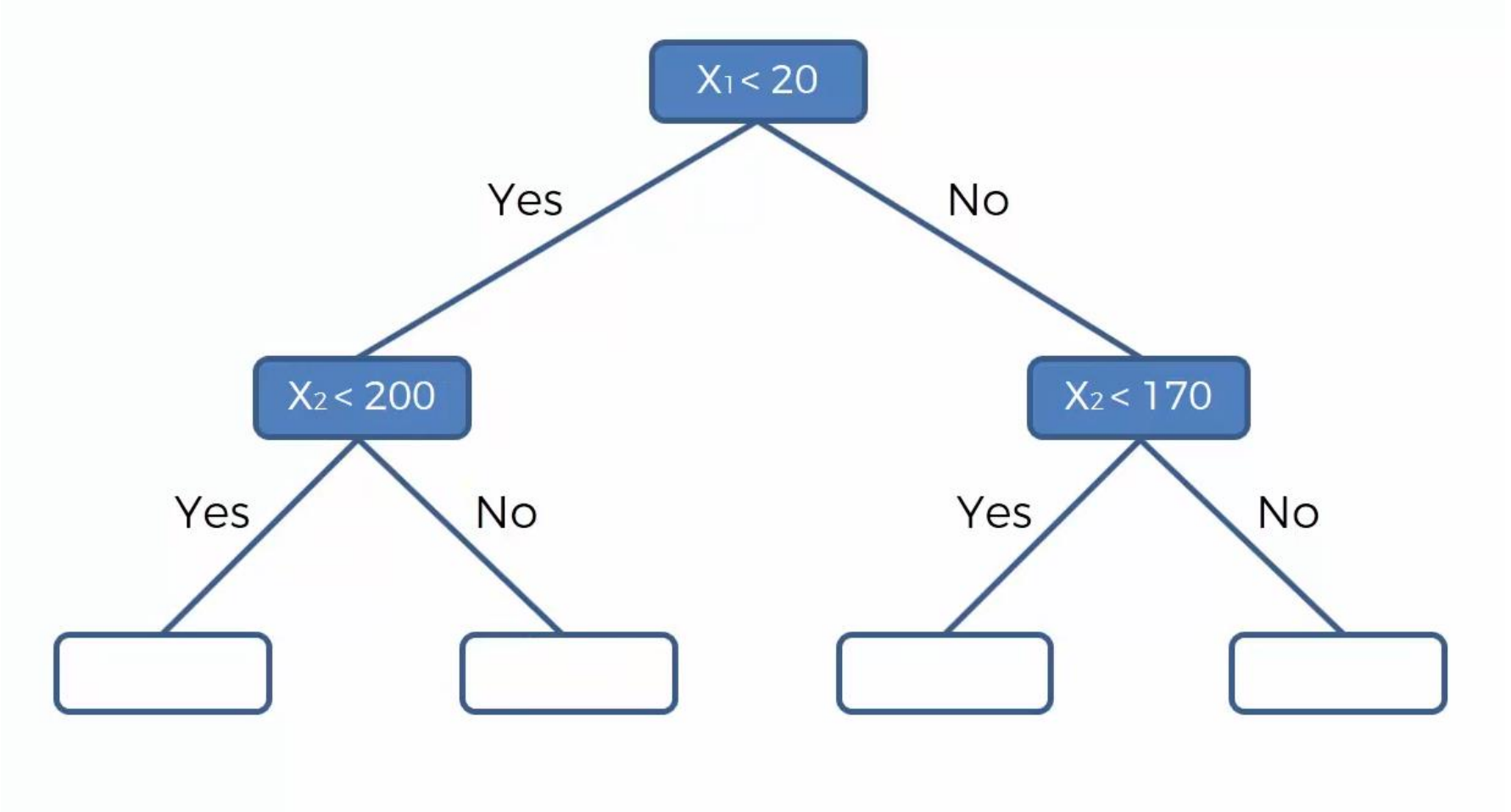


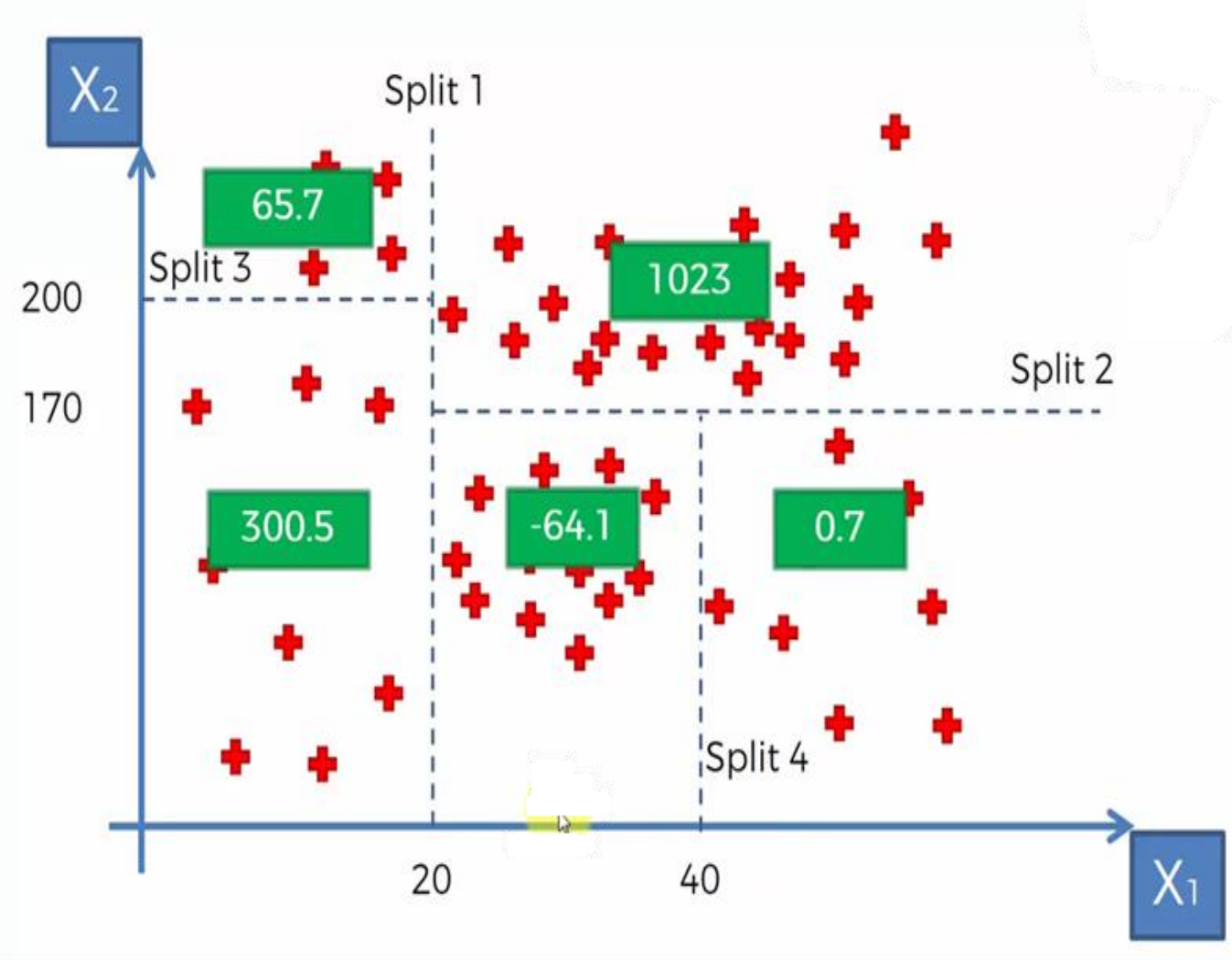
- Now what are we going to put in the empty boxes
- This is where we need to consider y i.e., our dependent variable that we need to predict or model
- What we need to check is how are we going to predict the value of y for let us assume an observation that has

$$x_1 = 30 \; and \; x_2 = 50$$



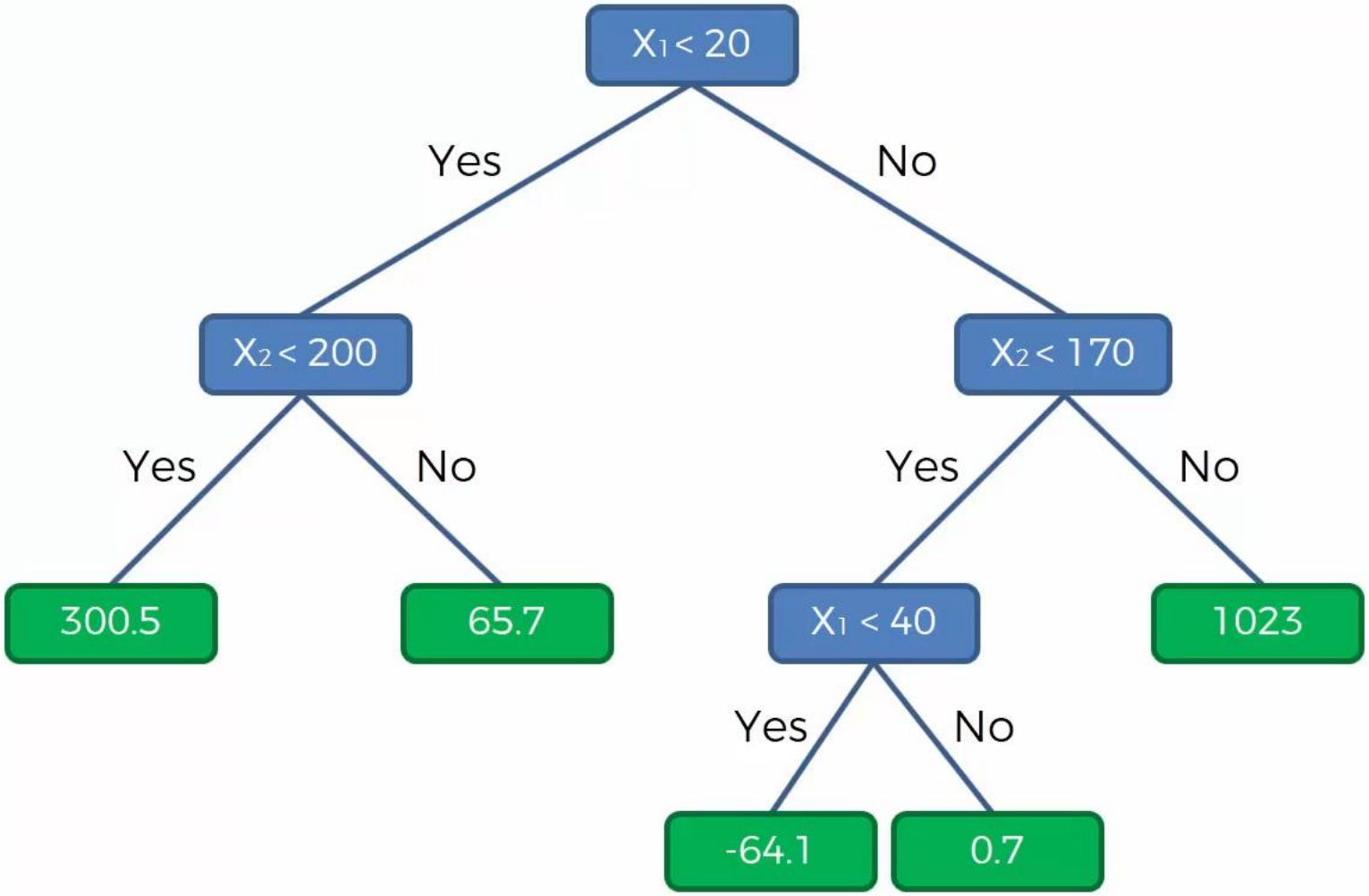
- The observation $x_1 = 30$ *and* $x_2 = 50$ lies in the terminal leaf given in green
- Now how does that information that the observation lies in the terminal leaf given in green help us in predicting the value of y.
- You take the average of all the values in your terminal leaf y


X1 < 20
Yes
No
X2 < 200
X2 < 170
Yes
No
Yes
No



- So let us assume the average for each terminal leaf are as given the figure.
- Then for the given point $x_1 = 30$ & $x_2 = 50$, the regression tree algorithm will give the output value or predicted value of y as -64.1
- It works on averages
- The goal is to add information to better predict y

- The last step is to add the mean values into the decision tree



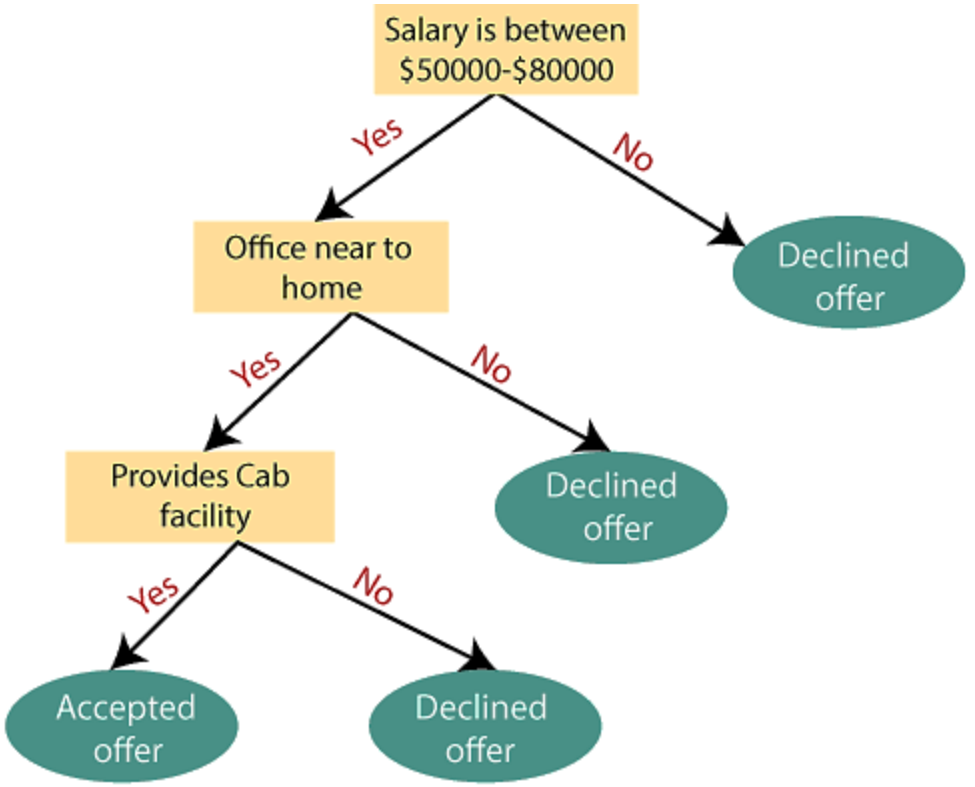
EXAMPLE OF DECISION TREE REGRESSION

The core algorithm for building decision trees called ID3 by J. R. Quinlan which employs a top-down, greedy search through the space of possible branches with no backtracking.

The ID3 algorithm can be used to construct a decision tree for regression by replacing Information Gain with Standard Deviation Reduction.

EXAMPLE

- Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not.
- To solve this problem, the decision tree starts with the root node (Salary attribute)
- The root node splits further into the next decision node (distance from the office)
- The next decision node further gets split into one decision node (Cab facility)
- Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer)



- The main issue arises that how to select the best attribute for the root node and for sub-nodes.
- To solve such problems there is an approach which is called as **Attribute selection measure (ASM).**
- By this measurement, we can easily select the best attribute for the nodes of the tree.
- There are two popular techniques for ASM, which are:

  (1) Entropy (2) Information gain

Entropy:

Entropy is a metric to measure the impurity in a given attribute.

Entropy can be calculated as:

- S= sample

$$H(S) = -p_1 \log_2(p_1) - p_2 \log_2(p_2)$$

p_1 is the proportion of class 1, p_2 is the proportion of class 2.

Example: Predicting Loan Customers

- Imagine a bank wants to classify customers as "high-risk" or "low-risk" based on their financial history.
- If the dataset has 50 customers, and 25 are high-risk (1) and 25 are low-risk (0)

$$H(S) = -\left(\frac{25}{50}\log_2\left(\frac{25}{50}\right) + \frac{25}{50}\log_2\left(\frac{25}{50}\right)\right) = 1$$

- The entropy is maximum because there is an equal split between the two classes.
- The dataset is highly uncertain.
- The value of Entropy is ranging from 0 to 1
- Higher value of Entropy leads to low accuracy
- Lower value of Entropy rises the accuracy
- In the above example, value is maximum means there is high risk

Information Gain (IG)

- Measures the reduction in entropy after a dataset is split on a particular feature
- In decision trees, the goal is to find the feature that maximizes information gain, i.e., the feature that gives the most reduction in uncertainty about the class.
- IG is computed as:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

- S is the original dataset
- A is the feature on which the dataset is split
- H(S) is the entropy of the dataset before the split
- H(Sv) is the entropy of the subsets after the split on feature A.

Example: Predicting Student Performance

- Consider a school predicting whether students will pass or fail based on features like "hours studied" and "attendance."
- Initially, the data may have equal instances of passing and failing students, leading to high entropy
- Splitting on "attendance" could give a large information gain.
- For example, if 80% of students with high attendance pass, the entropy will decrease significantly
- The information gain will be high.
- Therefore, the decision tree will prioritize "attendance" as the first split.

EXAMPLE: PREDICTION OF TENNIS TO PLAY OR NOT

Step-by-step construction of Decision Tree for the dataset

- **Step 1: Calculate the Initial Entropy of the Dataset**
- We have the following results:
- 9 instances of "Yes" (tennis is played)
- 5 instances of "No" (tennis is not played)

The entropy for the dataset is:

$$H(S) = -\left(\frac{9}{14}\log_2\frac{9}{14} + \frac{5}{14}\log_2\frac{5}{14}\right)$$

$$H(S) \approx -(0.643 \times \log_2 0.643 + 0.357 \times \log_2 0.357)$$

$$H(S) \approx 0.940$$

Day	Outlook	Temp	Humidity	Wind	Tennis?
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Step 2: Calculate Information Gain for Each Attribute

We now calculate the **information gain** for each attribute: **Outlook**, **Temperature**, **Humidity**, and **Wind**.

- **2.1 Information Gain for Outlook:**
- Outlook has 3 values: **Sunny**, **Overcast**, and **Rain**. Determine Entropy for each value.
- **Sunny**: 5 samples (3 "No", 2 "Yes")
- **Overcast**: 4 samples (all "Yes")
- **Rain**: 5 samples (2 "No", 3 "Yes")
- The weighted entropy for **Outlook** is:

$$I(\text{Sunny}) = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right) \approx 0.971$$

$$I(\text{Overcast}) = 0 \quad \text{(all positive)}$$

$$I(\text{Rain}) = -\left(\frac{2}{5}\log_2\frac{2}{5} + \frac{3}{5}\log_2\frac{3}{5}\right) \approx 0.971$$

$$\text{Weighted Entropy (Outlook)} = \frac{5}{14} \times 0.971 + \frac{4}{14} \times 0 + \frac{5}{14} \times 0.971$$

$$\approx 0.693$$

- The information gain for **Outlook** is:

$$\text{Gain}(T, \text{Outlook}) = 0.940 - 0.693 = 0.247$$

- **2.2 Information Gain for Temperature:**
- Temperature has 3 values: **Hot**, **Mild**, and **Cool**.
- **Hot**: 4 samples (2 "No", 2 "Yes")
- **Mild**: 6 samples (2 "No", 4 "Yes")
- **Cool**: 4 samples (1 "No", 3 "Yes")

$$I(\text{Hot}) = -\left(\frac{2}{4}\log_2\frac{2}{4} + \frac{2}{4}\log_2\frac{2}{4}\right) = 1.0$$

$$I(\text{Mild}) = -\left(\frac{2}{6}\log_2\frac{2}{6} + \frac{4}{6}\log_2\frac{4}{6}\right) \approx 0.918$$

$$I(\text{Cool}) = -\left(\frac{1}{4}\log_2\frac{1}{4} + \frac{3}{4}\log_2\frac{3}{4}\right) \approx 0.811$$

- The total entropy for **Temperature** is:

$$\text{Weighted Entropy (Temperature)} = \frac{4}{14} \times 1.0 + \frac{6}{14} \times 0.918 + \frac{4}{14} \times 0.811$$

$$\approx 0.911$$

- The information gain for **Temperature** is:

$$\text{Gain}(T, \text{Temperature}) = 0.940 - 0.911 = 0.029$$

2.3 Information Gain for Humidity:

- Humidity has 2 values: **High** and **Normal**.
- **High**: 7 samples (4 "No", 3 "Yes")
- **Normal**: 7 samples (1 "No", 6 "Yes")

$$I(\text{High}) = -\left(\frac{4}{7}\log_2\frac{4}{7} + \frac{3}{7}\log_2\frac{3}{7}\right) \approx 0.985$$

$$I(\text{Normal}) = -\left(\frac{1}{7}\log_2\frac{1}{7} + \frac{6}{7}\log_2\frac{6}{7}\right) \approx 0.592$$

- The total entropy for **Humidity** is:

$$\text{Weighted Entropy (Humidity)} = \frac{7}{14} \times 0.985 + \frac{7}{14} \times 0.592$$

$$\approx 0.789$$

- The information gain for **Humidity** is:

$$\text{Gain}(T, \text{Humidity}) = 0.940 - 0.789 = 0.151$$

2.4 Information Gain for Wind:

- Wind has 2 values: **Weak** and **Strong**.
- **Weak**: 8 samples (2 "No", 6 "Yes")
- **Strong**: 6 samples (3 "No", 3 "Yes")
- The weighted entropy for **Wind** is:

$$I(\text{Weak}) = -\left(\frac{2}{8}\log_2\frac{2}{8} + \frac{6}{8}\log_2\frac{6}{8}\right) \approx 0.811$$

$$I(\text{Strong}) = -\left(\frac{3}{6}\log_2\frac{3}{6} + \frac{3}{6}\log_2\frac{3}{6}\right) = 1.0$$

$$\text{Weighted Entropy (Wind)} = \frac{8}{14} \times 0.811 + \frac{6}{14} \times 1.0$$

$$\approx 0.892$$

- The information gain for **Wind** is:

$$\text{Gain}(T, \text{Wind}) = 0.940 - 0.892 = 0.048$$

Step 3: Choose the Best Attribute for the Root Node

The information gains for each attribute are:

- **Outlook**: 0.247
- **Temperature**: 0.029
- **Humidity**: 0.151
- **Wind**: 0.048

Since **Outlook** has the highest information gain (0.247), we choose **Outlook** as the root node.

Step 4: Split the Dataset on Outlook

4.1 Subtree for Outlook = Overcast:

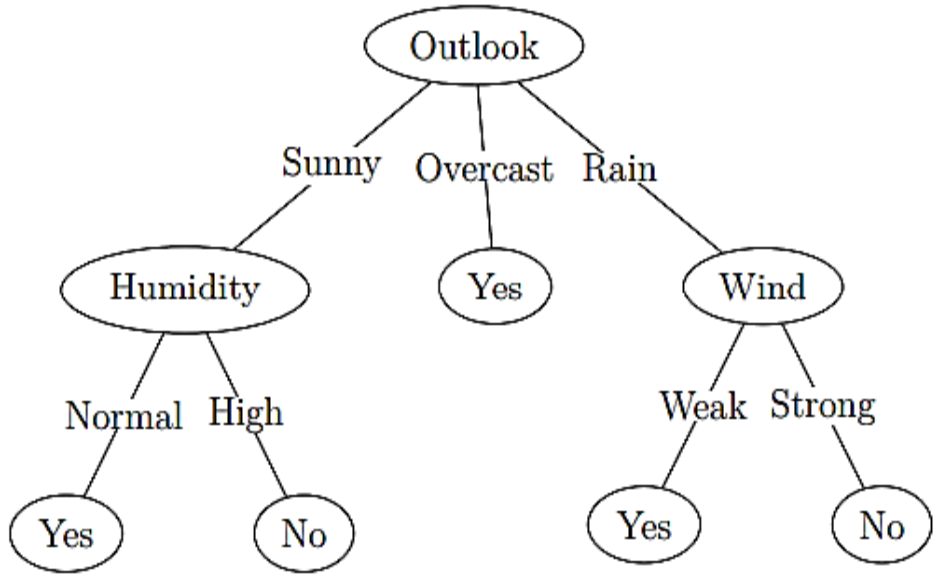
- All examples are "Yes", so this is a leaf node with label **Yes**.

4.2 Subtree for Outlook = Sunny:

- Now we have 5 samples (3 "No", 2 "Yes"). We repeat the process for these samples using the remaining attributes (**Humidity**, **Wind**).
- **Humidity** gives the highest information gain for this subset (computed earlier).
- If **Humidity = High**, then the label is **No** (since all instances with high humidity are "No").
- If **Humidity = Normal**, then the label is **Yes**.

4.3 Subtree for Outlook = Rain:

- For the subset where **Outlook = Rain**, we have 5 samples (2 "No", 3 "Yes").
- The attribute with the highest information gain here is **Wind**.
- If **Wind = Weak**, then the label is **Yes**.
- If **Wind = Strong**, then the label is **No**.



Prediction of the DT: There are 3 Yes and 2 No, Tennis will be played

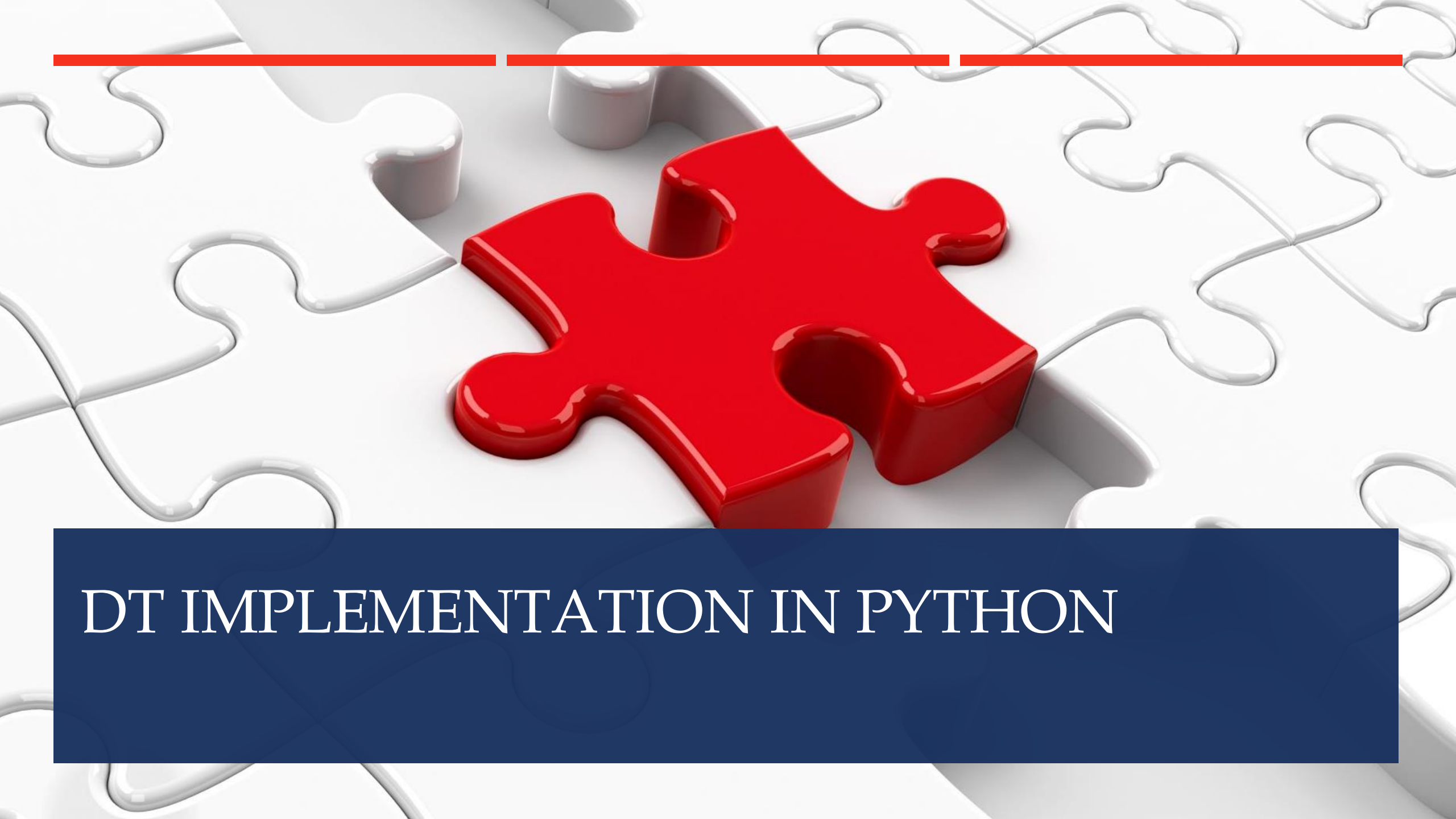
PROS AND CONS OF DT

Pros

- Easy to understand and interpret, making them accessible to non-experts.
- Handle both numerical and categorical data without requiring extensive preprocessing.
- Provides insights into feature importance for decision-making.
- Applicable to both classification and regression tasks.

Cons

- Potential for overfitting
- Sensitivity to small changes in data, limited generalization if training data is not representative
- Potential bias in the presence of imbalanced data.



DT IMPLEMENTATION IN PYTHON

CONTENTS

INTRODUCTION

We are using the same dataset of employee positions and salary

The problem is same that we need to predict the salary of the perspective new employee

The decision tree regression algorithm is not very well adapted to these simple datasets.

It is usually useful for datasets involving multiple independent variables.

We do not need to apply feature scaling for the decision tree algorithm

PROBLEM AT HAND

Position	Level	Salary
Business Analyst	1	45000
Junior Consultant	2	50000
Senior Consultant	3	60000
Manager	4	80000
Country Manager	5	110000
Region Manager	6	150000
Partner	7	200000
Senior Partner	8	300000
C-level	9	500000
CEO	10	1000000

- The old dataset 'Position_Salaries.csv' and old problem of salary will be used

IMPORTING THE LIBRARIES AND DATASET

```
# importing the libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# import the dataset
dataset = pd.read_csv('Position_Salaries.csv')
X = dataset.iloc[:,1:-1].values
y = dataset.iloc[:,-1].values
```

Once you have imported the dataset check to see which columns you need to consider

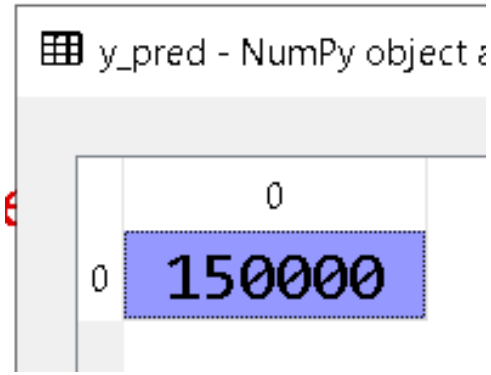
Also check whether you need to apply any data pre-processing techniques

Handling missing values	Data cleaning	Encoding categorical data	Feature scaling

TRAINING THE DECISION TREE REGRESSION MODEL

```
# training the decision tree regression model
from sklearn.tree import DecisionTreeRegressor
regressor = DecisionTreeRegressor(random_state=0)
regressor.fit(X,y)
```

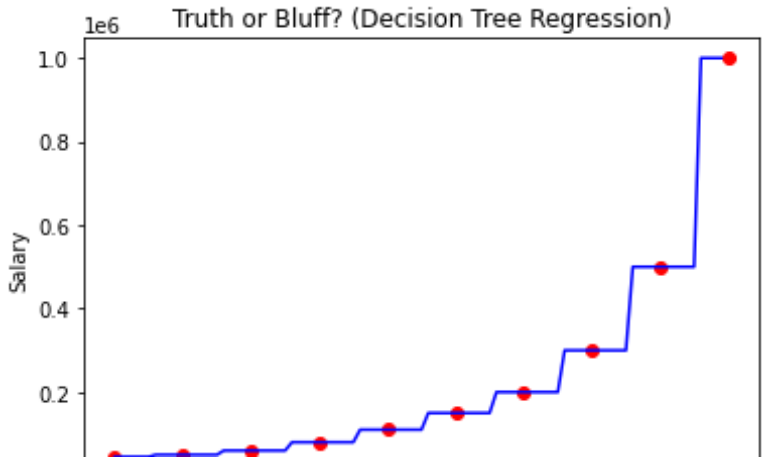
- Congragulations!!! You have trained your first decision tree regression model. Well done…
- Predicting the new value?
  - Please do this yourself




```
# predicting the new result
y_pred = regressor.predict([[6.5]])
print(y_pred)
```

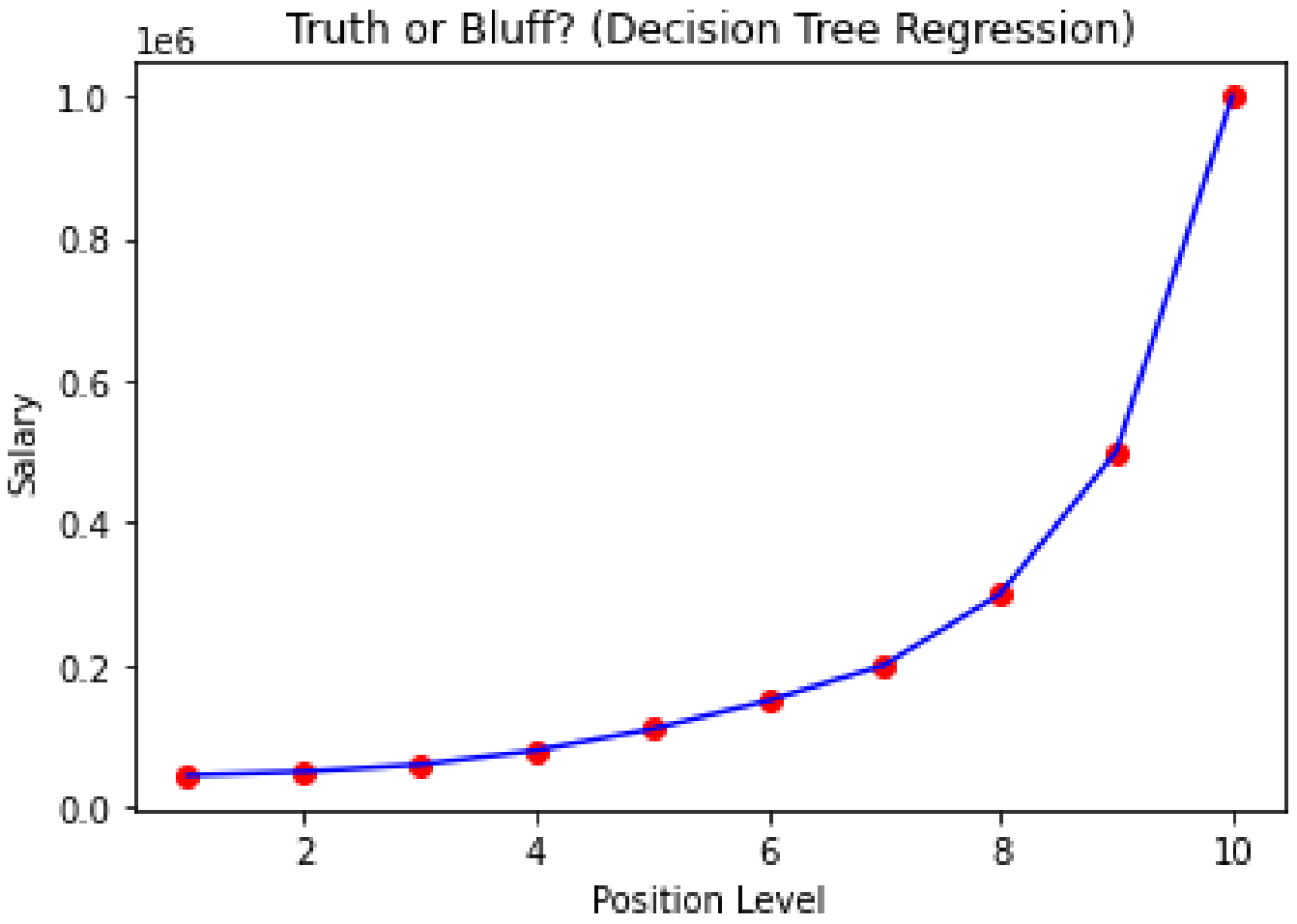
- The predicted salary is lower than the requested salary.

PLOTS FOR THE RESULTS




```
laizing the results in high resoluti
 = np.arange(min(X),max(X),0.1)
 = x_grid.reshape(len(x_grid),1)
atter(X, y, color='red')
ot(x_grid, regressor.predict(x_grid)
tle('Truth or Bluff? (Decision Tree
abel('Position Level')
abel('Salary')
ow()
```

- The decision tree regression is not very well adapted to two dimensional datasets i.e., one independent and one dependent variable
- The staircase curve is due to the splitting of nodes.
- We can observe that 0.5 point before and after a position level gives the same salary
- This is the average between the two levels
- The decision tree plot is not continuous, rather it is discrete in the sense that we jump from one value to the next in each step.



LOW RESOLUTION PLOT, NOT CORRECT

WRONG!!!

EVALUATE THE MODEL

We will not evaluate the model here since in training we can see all the datapoints pass through the curve.

Therefore, for training

RMSE will be zero

Adjusted R score will be 1

Since RMSE will be zero so Durbin Watson statistics will be ‘NaN’

CLASS EXERCISE

- Please try this decision tree regression for higher dimensional datasets i.e., the one used in multiple linear regression now.
- Once you have trained the model for higher dimensional dataset use the given instance to predict the 'Profit'
  - State = 'Florida' (after one hot encoding the code is 010)
  - R&D spent = 10
  - Administration = 9194
  - Marketing = 2000

  - [[0,1,0,10,9194,2000]]

QUESTIONS